

Modul Media Programming, Lehrveranstaltung AV Programming



Prof. Dr. Eva Wilk

HAW Hamburg, Fakultät INF, Studiengang Medieninformatik

Test

PSO MI und MS:

In jedem Modul mit Prüfungsform Klausur (K) können zusätzlich bis zu zwei Tests nach § 14 Absatz 3 Punkt 11 APSO-INGI geschrieben werden, deren Ergebnisse in der Summe mit bis zu 20% in die Klausurnote eingehen können.

§ 14 Absatz 3 Punkt 11 APSO-INGI:

Der Test ist eine schriftliche Arbeit, in dem die Studierenden nachweisen, dass sie Aufgaben zu einem klar umgrenzten Thema unter Klausurbedingungen bearbeiten können. Die Dauer eines Tests beträgt mindestens 15, höchstens 90 Minuten. In studiengangsspezifischen Prüfungs- und Studienordnungen kann bestimmt werden, dass die Einzelergebnisse der Tests mit in die Bewertung der Klausuren einbezogen werden.

Test 1 am 2.6.2026: Dauer: 20 Minuten für Media Programming gesamt

- Taxonomiestufe: Anwenden
- erlaubte Unterlagen: keine
- Taschenrechner: ja
- Fragen zum Anwenden des bis dahin Behandelten
- Programmieren: nicht am Rechner, aber (Pseudo-)Code schreiben / ergänzen / korrigieren

Thema 3 – Audioformate und –datenreduktion

Lernziel

Die Studierenden wählen für einfache medieninformatische Szenarien geeignete Audioformate und Codierungsverfahren aus, begründen diese Auswahl fachlich und setzen einfache Formatinspektionen sowie Konvertierungen mit Python und punktuell mit FFmpeg exemplarisch um.

Aufgabe 7

Wählen Sie für jedes Szenario ein Format und begründen Sie die Wahl:

1. Archivierung eines hochwertigen Interviews
2. Veröffentlichung eines Podcasts
3. Python-basierte Analyse im Kurs
4. Versand einer größeren Audiosammlung bei knappem Speicher

Zeit: 8 Minuten

Ergebnisse: a. Format nennen, b. zwei Argumente, c. ein möglicher Nachteil

Thema 3 - Audioformate und Audio-Datenreduktion

Übersicht

- PCM
- WAV
- FLAC
- MP3
- AAC
- ffprobe / ffmpeg
- Übungsaufgaben zum 2.6.2026

Repräsentation, Format, Codec

- PCM - *Pulse Code Modulation*. Digitale Repräsentation von Abtastwerten
- WAV - *Waveform Audio File Format*. Datei-Format / Container. Enthält PCM-Daten.
- FLAC - *Free Lossless Audio Codec*. Verlustfreier Codec (**Coder-Decoder**)
- „MP3“ - *MPEG 1 Audio Layer III*. Verlustbehafteter perceptual codec
- AAC – *Advanced Audio Coding*. Verlustbehafteter perceptual codec

Kurz zusammengefasst:

WAV/PCM: transparent, gut inspizierbar

FLAC: weniger Daten als WAV, aber bitgenau rekonstruierbar

MP3/AAC: weniger Daten als WAV, aber nicht bit-identisch zum Original

Container und Codec unterscheiden

1. Audiosignal, z. B. Sprache, Musik, Geräusch.

Wird digital repräsentiert in Form von →

2. Audiodaten bzw. Codierung

PCM = Abtastwerte in binärer Darstellung

FLAC = verlustfrei codierte Audiodaten

MP3, AAC, Vorbis = verlustbehaftet codierte Audiodaten

wie die Daten in der Datei abgelegt werden, organisiert →

3. Container bzw. Dateiformat

WAV

Ogg

Matroska

M4A / MP4

Warum ist „WAV ist unkomprimiert“ fachlich weniger präzise als „WAV enthält PCM-Daten“?

Datei-Endung ≠ Dateiformat ≠ Codec

Endung: .wav, .ogg, .m4a

Container/Dateiformat: WAV, Ogg, Matroska, MP4/M4A

Codec: PCM, Vorbis, MP3, AAC, FLAC

Beispiele:

musik.wav = WAV + PCM

musik.ogg = Ogg + Vorbis

archiv.mka = Matroska + Audio-Codec

podcast.m4a = MP4/M4A + AAC

Der Container organisiert die Datei.

Der Codec bestimmt, wie die Audiodaten dargestellt oder komprimiert sind.

Beispiel 10: WAV/PCM praktisch verstehen

```
from pathlib import Path
import wave

datei = "beispiel.wav"

with wave.open(datei, "rb") as wf:
    kanaele = wf.getnchannels()
    samplebreite = wf.getsampwidth()
    samplerate = wf.getframerate()
    frames = wf.getnframes()

dauer = frames / samplerate
print(kanaele, samplebreite, samplerate, frames, dauer)
print("Dateigröße:", Path(datei).stat().st_size)
```

Wie ändert sich die Dateigröße, wenn gespeichert wird mit:

- Mono statt Stereo
- 8 Bit statt 16 Bit pro Abtastwert
- Abtastfrequenz 22,05 kHz

Erläuterung

- `wave` liest WAV-Metadaten, nicht das Audiosignal selbst
- `getnchannels()` liefert die Kanalzahl, also z. B. Mono oder Stereo
- `getsampwidth()` gibt die Wortbreite in Bytes zurück; 2 Bytes entsprechen 16 Bit
- `getframerate()` ist die Abtastfrequenz, also z.B. 44100 Hz
- `getnframes()` ist die Zahl der Abtastwerte pro Kanal
- `Dauer der Datei = frames : samplerate`
- `Path(...).stat().st_size`: Dateigröße wird aus dem Dateisystem ausgelesen

Ausgabe

- Datei: beispiel.wav
- Kanäle: 2
- Wortbreite (Bytes): 2
- Samplerate (Hz): 44100
- Frames: 11868780
- Dauer (s): 269.1333333333333
- Datenmenge (Bytes): 47475120.0
- Dateigröße (Bytes): 47475400

```

Datei: beispiel.wav
Kanäle: 2
Wortbreite (Bytes): 2
Samplerate (Hz): 44100
Frames: 11868780
Dauer (s): 269.1333333333333
Datenmenge (Bytes): 47475120.0
Dateigröße (Bytes): 47475400

```

Warum ist die reale Dateigröße nicht exakt identisch zur berechneten Dateigröße?

Lossless Coding

- Ziel: kleinere Datei
- Bedingung: kein Informationsverlust
- Beispiel: FLAC
- Nach dem Decoding: wieder ursprüngliche Audiodaten

https://www.rfc-editor.org/rfc/rfc9639.html?utm_source=chatgpt.com

Prädiktor-Korrektor-Verfahren

Encoder: mit einem expliziten Verfahren wird aus u_n ein Prädiktor u_{n+1}^* für u_{n+1} geschätzt. Der Fehler $e = u_{n+1}^* - u_{n+1}$ wird ermittelt.

→ e wird übertragen →

Decoder: u_{n+1}^{**} wird aus u_n geschätzt und durch e korrigiert.

https://www.rfc-editor.org/rfc/rfc9639.html?utm_source=chatgpt.com

Beispiel 11: Lossless coding

```
from pathlib import Path
import numpy as np
import soundfile as sf

sr = 44100
dauer_s = 3.0
t = np.linspace(0, dauer_s, int(sr * dauer_s), endpoint=False)

signal = (
    0.5 * np.sin(2 * np.pi * 220 * t) + 0.2 * np.sin(2 * np.pi * 440 * t)
).astype(np.float32)

sf.write("signal.wav", signal, sr, subtype="PCM_16")
sf.write("signal.flac", signal, sr, subtype="PCM_16")

wav_size_bytes = Path("signal.wav").stat().st_size
flac_size_bytes = Path("signal.flac").stat().st_size
```

Warum ist FLAC kleiner als WAV?

Warum ist mono_8bit.wav fachlich etwas anderes als signal.flac?

Aufgabe 8

Variieren Sie die

- Signaldauer
- Signalart
- Mono → Stereo
- Lautstärke

Verwenden Sie eine Sprachdatei und/oder eine Musikdatei

Zeit: 8 Minuten

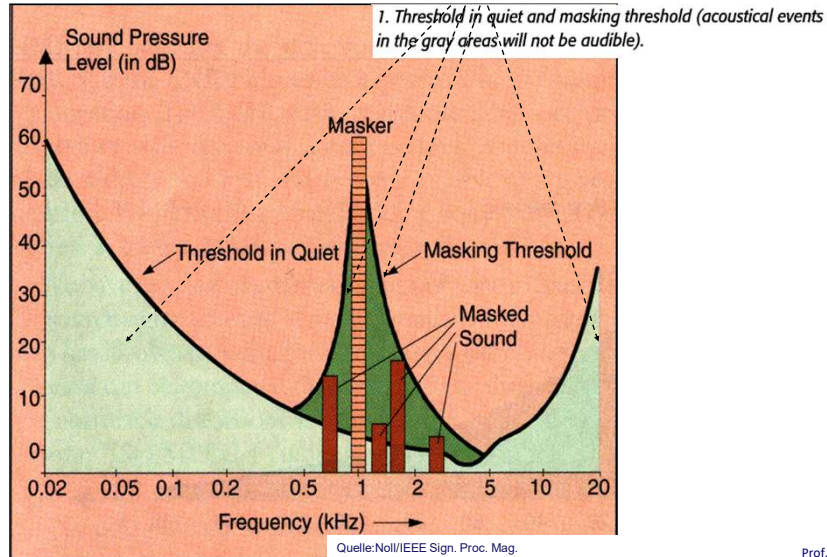
Lossy Coding

- Nicht bit-identisch zum Original
- Ziel: starke Reduktion der Datenmenge
- Grundidee: $\text{Datenrate} = \text{anz. kanäle} \cdot \text{bits pro Abtastwert} \cdot \text{Abtastfrequenz}$

Joint Stereo Coding

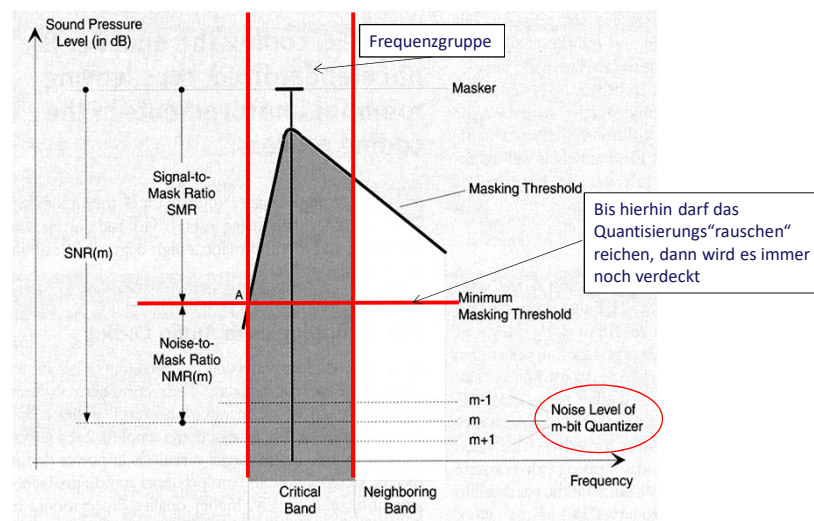
- Zeit-/Frequenzanalyse
 - psychoakustisches Modell
 - Quantisierung/Codierung
- Beispiele:
 - MP3
 - AAC

Lossy Coding



Prof. Dr. E. Wilk
HAW Hamburg

Lossy Coding



Quelle: Noll/IEEE Sign. Proc. Mag.

Prof. Dr. E. Wilk
HAW Hamburg

Lossy Coding

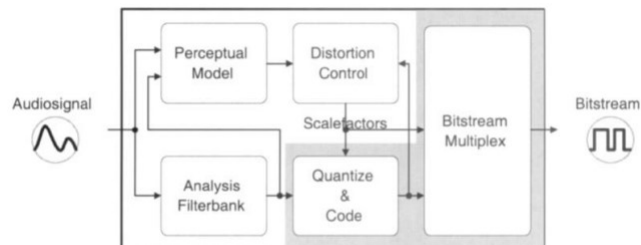


Figure 2: Block diagram of MPEG-2 AAC encoder

Neubauer, Herre: Audio Watermarking of MPEG-2 AAC Bit Streams
https://www.iis.fraunhofer.de/content/dam/iis/de/doc/ame/conference/AES-108-Convention_Audio_WatermarkingofMPEG-2-AACBitstreams_AES101.pdf

Prof. Dr. E. Wilk
 HAW Hamburg

Gegenüberstellung von MP3, AAC, FLAC, WAV

Format	Typ	Stärke	Schwäche	Typischer Einsatz
WAV	nicht datenreduziert	transparent	groß	Lehre, Analyse, Profi-Aufnahme und -Bearbeitung
FLAC	verlustfrei datenreduziert	kleiner	weniger universell als MP3	Archivierung
MP3	verlustbehaftet datenreduziert	weit verbreitet	Qualitätsverlust	Distribution
AAC	verlustbehaftet datenreduziert	effizienter als MP3	Qualitätsverlust	Streaming, Distribution

Wann würden Sie AAC statt MP3 wählen?“

FFmpeg

- FFmpeg ist Werkzeugkasten für Audio und Video: „universal media converter“
- Funktionen:
 - lesen, analysieren, umwandeln, filtern von Medien
 - funktioniert über die Kommandozeile
 - Einsatz für viele Formate und Codecs
 - Datei / Stream → FFmpeg → neue Datei / neues Format
Beispiel: `ffmpeg -i beispiel.wav beispiel.flac`
 - in diesem Kurs wichtig für:
 - Formate verstehen
 - Sichtbarmachen von Formaten, Codecs, Verarbeitungsschritten
 - Dateien prüfen
 - Audio/Video konvertieren
- `ffmpeg` Konvertierungs-Tool, konvertiert und verarbeitet Medien
- `ffprobe` Teil von `ffmpeg`, Analyse-Tool, zeigt Informationen über Datei und Streams

https://www.ffmpeg.org/ffmpeg.html?utm_source=chatgpt.com

ffprobe

- `ffprobe` ist ein externes Kommandozeilenprogramm aus der FFmpeg-Toolchain
 - FFmpeg selbst stellt nur den Source Code bereit, für fertige ausführbare Dateien wird auf vorkompilierte Pakete/Builds verwiesen
- Windows-Version von FFmpeg installieren, und dann:
- a. den Ordner mit `ffprobe.exe` zum PATH hinzufügen, oder
 - b. im Python-Code den vollen Pfad zu `ffprobe.exe` angeben.
- `ffprobe` liest aus Mediendateien die
 - Containerinformationen,
 - Streaminformationen,
 - Codec-Angaben,
 - Bitraten,
 - ...

Beispiel 12: FFmpeg und ffprobe aus Python aufrufen

```
from pathlib import Path
import subprocess

eingabe = "signal.wav"

subprocess.run(["ffmpeg", "-y", "-i", eingabe, "signal.flac"])
subprocess.run(["ffmpeg", "-y", "-i", eingabe, "-b:a", "128k", "signal_128k.mp3"])
subprocess.run(["ffmpeg", "-y", "-i", eingabe, "-b:a", "64k", "signal_64k.mp3"])

for datei in ["signal.wav", "signal.flac", "signal_128k.mp3", "signal_64k.mp3"]:
    groesse = Path(datei).stat().st_size
    print(datei, ":", groesse, "Bytes =", groesse * 8, "bit")

subprocess.run(["ffprobe", "-hide_banner", "-show_format", "-show_streams", "signal_128k.mp3"])
ergebnis = subprocess.run(
    ["ffprobe", "-hide_banner", "-show_format", "-show_streams", "signal_128k.mp3"],
    capture_output=True,
    text=True
)

print("\n Ergebnis der Analyse von signal_128k.mp3 mit ffprobe:")
print(ergebnis.stdout[:800])
```

Beispiel 12: FFmpeg und ffprobe aus Python aufrufen

- `eingabe = "signal.wav"` legt die Ausgangsdatei fest
- Der erste ffmpeg-Aufruf erzeugt aus der wav-Datei eine FLAC-Datei
- der zweite Aufruf erzeugt eine MP3-Datei mit der Bitrate 128kbit/s
- der dritte Aufruf erzeugt eine MP3-Datei mit der Bitrate 64kbit/s
- `-b:a` setzt die Audio-Bitrate.
Geringere Bitrate:
→ kleinere Datei
→ stärkerer verlustbehaftete Codierung = schlechtere Qualität
- in der Schleife werden anschließend die Dateigrößen ausgegeben

Beispiel 12: FFmpeg und ffprobe aus Python aufrufen

...

- `ffprobe` untersucht die Datei `signal_128k.mp3` und gibt Format- und Streaminformationen aus
- `capture_output=True` sorgt dafür, dass die Ausgabe nicht nur im Terminal erscheint, sondern von Python abgefangen wird
- `text=True` sorgt dafür, dass die Ausgabe als lesbarer Text vorliegt
- `ergebnis.stdout` enthält die Analyse-Ausgabe
- `[ :800]` kürzt die Ausgabe, damit auf der Folie oder in der Konsole nicht sofort zu viel Text erscheint

ffprobe - Anleitung

1. FFmpeg herunterladen, als Windows-Build <https://www.ffmpeg.org/download.html>
2. ZIP entpacken
3. Prüfen, ob **ffprobe.exe** dabei ist mit `C:\ffmpeg\bin\ffprobe.exe`
4. Testen mit:

```
import subprocess
ffprobe_path = r"C:\ffmpeg\bin\ffprobe.exe"
result = subprocess.run(
    [ffprobe_path, "-version"],
    capture_output=True, text=True
)
print(result.stdout)
```

5. Wenn das klappt, den Pfad `C:\ffmpeg\bin` der Windows-Umgebungsvariable PATH hinzufügen:
Win + R drücken und eingeben: `SystemPropertiesAdvanced`. Dort den Pfad `C:\ffmpeg\bin` ergänzen.
 Prüfen mit:

```
import subprocess

subprocess.run(["ffprobe", "-version"], check=True)
if subprocess: print ("läuft")
```

Einsatz von shutil, subprocess, json

Diese drei Bibliotheken werden nicht für die Audiosignalverarbeitung selbst benötigt, sondern für den Zugriff auf ein externes Analysewerkzeug.

`ffprobe` liefert die Mediendaten,

`subprocess` startet es,

`json` liest die strukturierte Ausgabe ein, und

`shutil` prüft, ob das Programm überhaupt gefunden wird.

Einsatz von shutil, subprocess, json

shutil

- ist eine Standardbibliothek für Datei- und Systemhilfsfunktionen
- kann Dateien kopieren, Verzeichnisse verwalten, Programme im Systempfad suchen
- Beispiel:

```
shutil.which("ffprobe")
```

→ prüft, ob ein Programm `ffprobe` im PATH gefunden wird.

Einsatz von shutil, subprocess, json

subprocess

- dient dazu, andere Programme aus Python heraus zu starten
- `ffprobe` ist ein externes Programm, kein Python-Modul
- hier: Python liest die Formatinformationen nicht selbst aus, sondern lässt das `subprocess` tun und holt sich die Ausgabe zurück:

```
result = subprocess.run(
    ["ffprobe", "-version"],
    capture_output=True,
    text=True
)
```

Quellen zu Thema 3

FFmpeg Documentation <https://ffmpeg.org/ffmpeg.html>

Stichworte: einfache Konvertierungen zwischen WAV, FLAC, MP3 und AAC

FFprobe Documentation <https://ffmpeg.org/ffprobe.html>

Stichworte: Inspektion von Format- und Streaminformationen

Fraunhofer: MP3 and AAC Explained

https://www.iis.fraunhofer.de/content/dam/iis/de/doc/ame/conference/AES-17-Conference_mp3-and-AAC-explained_AES17.pdf

Stichworte: perceptual coding, Unterschiede zwischen MP3 und AAC, Blockdiagramm eines perceptual coders.

python-soundfile Documentation <https://python-soundfile.readthedocs.io/>

Stichworte: : WAV- und FLAC-Schreiben/Lesen in Python

Python wave <https://docs.python.org/3/library/wave.html>

Stichworte: WAV/WAVE, nicht-datenreduziertes PCM, Metadatenzugriff im Python-Beispielcode

RFC 9639: FLAC <https://www.rfc-editor.org/rfc/rfc9639.html>

Stichworte: Definition von FLAC als verlustfreiem Audiocodex; Grundlage für lossless coding.